

Google Cloud Fundamentals

Organization, projects, billing and identity — hands-on in Cloud Shell

Six chapters, five labs

- 01** What a cloud provider actually gives you
- 02** Getting to the keyboard
- 03** Where things live — the resource hierarchy
- 04** Who can do what — identity and IAM
- 05** Paying for it — billing, cost and quotas
- 06** Who did what — audit logs

01

What a cloud provider actually gives you

Which layers Google runs for you, which you still own, and where in the world your workload actually executes.

01 What a cloud provider actually gives you

02 Getting to the keyboard

03 Where things live — the resource hierarchy

04 Who can do what — identity and IAM

05 Paying for it — billing, cost and quotas

06 Who did what — audit logs

SHARED RESPONSIBILITY

Google secures the cloud; you secure what you put in it

Layer	IaaS (Compute Engine)	PaaS (Cloud Run, GKE Autopilot)	SaaS (BigQuery, Workspace)
Data, classification, retention	You	You	You
Identities & access policy (IAM)	You	You	You
Application code & config	You	You	Google
Guest OS, patches, runtime	You	Google	Google
Hypervisor, network fabric	Google	Google	Google
Physical data centre, hardware	Google	Google	Google

Everything in this course — projects, IAM, org policy, budgets, audit logs — lives in the top two rows. That is the part you own on every service.

THE RULE The provider never owns your data, your identities or your access decisions — on any service tier.

"It's on Google Cloud, so security is Google's problem"

WEAK

Team stores customer invoices in a Cloud Storage bucket.

"Google encrypts everything and runs the data centre, so we're covered."

Six months later a contractor's script uses a key with roles/editor and copies the bucket to a personal project. No alert fires. Nobody can say who did it.

STRONG

Same bucket, same encryption — plus the customer-side controls:

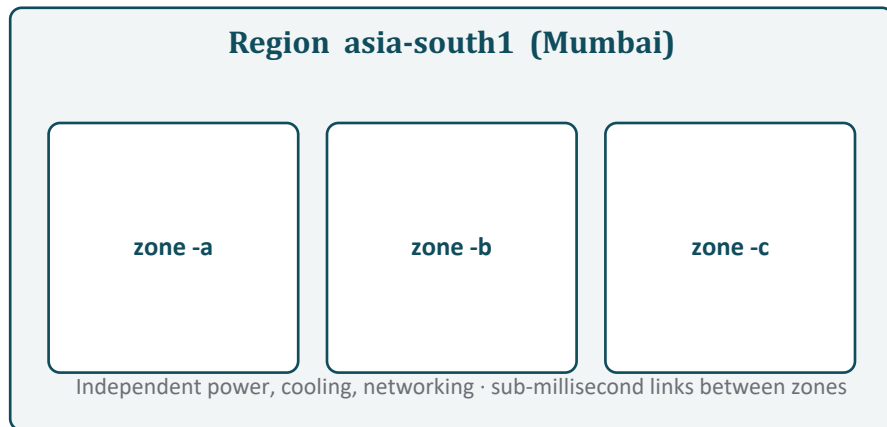
- Public access prevention enforced, uniform bucket-level access on
- Access via a group with roles/storage.objectViewer, no service-account keys
- Data Access audit logs enabled; a budget alert on egress

The copy attempt is denied, and the attempt itself appears in the audit log with the caller's identity.

THE RULE Google's encryption does not decide who may read the data. Your IAM policy does.

REGIONS AND ZONES

A region is a geography; a zone is a failure domain inside it



Resource scope	Survives...	Examples
Zonal	nothing outside its zone	VM instance, zonal persistent disk, zonal GKE cluster
Regional	loss of one zone	regional disk, Cloud Run service, Cloud SQL HA, regional GKE
Multi-regional	loss of a region	Cloud Storage multi-region bucket, Spanner multi-region
Global	n/a — control plane	VPC network, IAM, HTTP(S) load balancer, images

THE RULE Design for the zone to fail. Pick the resource scope that matches how much you can lose.

Four questions decide the region, then gcloud shows what's there

- Residency — where may the data legally sit?
- Latency — where are the users and the other systems?
- Services — is the product (and GPU/machine type) available there?
- Price — the same VM costs more in some regions
- Resilience — one region, or two for DR?

```
$ gcloud compute regions list --filter="name~^asia-south" \
--
format="table(name,status,zones.len():label=ZONES)"
NAME          STATUS  ZONES
asia-south1   UP      3
asia-south2   UP      3

$ gcloud compute zones list --filter="region~asia-south1$"
NAME          REGION    STATUS
asia-south1-a asia-south1 UP
asia-south1-b asia-south1 UP
asia-south1-c asia-south1 UP
```

gcloud · Cloud Shell

THE RULE Residency, then latency, then service availability, then price — in that order.

PREDICT THE OUTPUT

Will this attach?

```
$ gcloud compute disks create data-disk --zone=asia-south1-a --size=10GB
$ gcloud compute instances create web-1 --zone=asia-south1-b \
  --machine-type=e2-small --disk=name=data-disk
```

The disk is in zone a, the VM in zone b, same region.

Does the second command succeed? If not, what is the smallest change that makes it work — and what would you use if the VM must survive a zone outage?

Answer in the speaker notes — discuss before revealing.

02

Getting to the keyboard

Console, gcloud and client libraries are three doors into one API. Cloud Shell gives you all three with nothing to install.

01 What a cloud provider actually gives you

02 Getting to the keyboard

03 Where things live — the resource hierarchy

04 Who can do what — identity and IAM

05 Paying for it — billing, cost and quotas

06 Who did what — audit logs

Console, gcloud and client libraries all call the same endpoint

Console

Navigation menu → Cloud Storage → Buckets

Best for: exploring, one-off checks, learning what a resource's fields are called.

```
$ gcloud storage ls
gs://acme-store-invoices/
gs://acme-store-static/
```

Best for: scripts, CI, reproducible ops

```
from google.cloud import storage
client = storage.Client()
for b in client.list_buckets():
    print(b.name)
acme-store-invoices
acme-store-static
```

Best for: inside your application

Python

All three send GET <https://storage.googleapis.com/storage/v1/b?project=acme-store-dev> with your credentials — the response is identical.

THE RULE Learn the resource in the console, automate it with gcloud, embed it with the client library.

Anatomy of a command, and the properties every command reads

```
gcloud compute instances create web-1 --zone=asia-south1-a --machine-type=e2-small  
  ^group   ^subgroup ^verb   ^name   ^flags
```

```
$ gcloud config list  
[compute]  
region = asia-south1  
zone = asia-south1-a  
[core]  
account = dev1@acme-store.example  
project = acme-store-dev  
  
$ gcloud config set project acme-store-dev  
Updated property [core/project].
```

- Properties = defaults every command reads
- --project / --zone on a command override them
- Multiple configurations: gcloud config configurations list
- Help is built in: gcloud storage buckets create --help
- Every command has a release track: gcloud, gcloud beta, gcloud alpha

THE RULE Before any gcloud session: gcloud config list. Wrong project is the most common mistake in this course.

--format and --filter turn any command into a query

WEAK

Piping to grep and awk:

```
gcloud projects list | grep acme | awk '{print $3}'
```

Breaks when a column moves or a name contains a space. Not portable.

STRONG

Ask the API for the field:

```
gcloud projects list --filter="projectId:acme-*" --  
format="value(projectNumber)"
```

Stable field names, no parsing.

```
$ gcloud projects describe acme-store-dev --format=json  
{  
  "createTime": "2026-08-03T09:14:22.101Z",  
  "lifecycleState": "ACTIVE",  
  "name": "Acme Store (dev)",  
  "projectId": "acme-store-dev",  
  "projectNumber": "734912580216"  
}  
$ gcloud projects describe acme-store-dev --format="value(projectNumber)"  
734912580216
```

THE RULE Never parse gcloud tables. Use `--format=value(field)` and `--filter` with the JSON field names.

CLOUD SHELL

A free, pre-authenticated Linux VM in your browser — with everything installed

- Ephemeral e2-small VM, Debian, 5 GB persistent \$HOME
- Authenticated as you — no keys, no gcloud auth login
- gcloud, gsutil, kubectl, terraform, python3, node, docker preinstalled
- Built-in editor (Open Editor) and web preview on port 8080
- Times out when idle; your files survive, running processes don't

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to acme-
store-dev.
dev1@cloudshell:~ (acme-store-dev)$ gcloud --version
Google Cloud SDK 583.0.0
bq 2.1.30
core 2026.09.01
gsutil 5.35
dev1@cloudshell:~ (acme-store-dev)$ echo
$GOOGLE_CLOUD_PROJECT
acme-store-dev
dev1@cloudshell:~ (acme-store-dev)$ gcloud auth list
ACTIVE  ACCOUNT
*      dev1@acme-store.example
```

THE RULE Cloud Shell is the course environment: nothing to install, authenticated as you, same gcloud everyone else has.

Every service is off until its API is enabled in the project

BEFORE

```
$ gcloud compute regions list
```

ERROR: (gcloud.compute.regions.list) HTTPError 403: Compute Engine API has not been used in project 734912580216 before or it is disabled. Enable it by visiting <https://console.developers.google.com/apis/api/compute.googleapis.com/overview?project=734912580216> then retry.

AFTER

```
$ gcloud services enable compute.googleapis.com
```

Operation "operations/acat.p2-734912580216-8f3e..." finished successfully.

```
$ gcloud compute regions list --limit=1
NAME      STATUS
africa-south1  UP
```

```
$ gcloud services list --available --filter="name:storage" --format="value(config.name)"
storage.googleapis.com
$ gcloud services list --enabled --format="value(config.name)" | wc -l
23
```

THE RULE "API not enabled" is not a permission problem. Enable the service, then retry.

First working project

HANDS-ON · LAB 1

Scenario: Northwind Logistics has given your team a fresh sandbox project for a new 'shipments' service. Make it usable.

- 1 Open Cloud Shell. Run `gcloud config list` and confirm `core/project` is YOUR sandbox project.
- 2 `export PROJECT_ID=$(gcloud config get-value project)` and echo it.
- 3 `gcloud projects describe $PROJECT_ID` — note `projectNumber` and `lifecycleState`.
- 4 Store the number: `export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")`
- 5 Enable the APIs: `gcloud services enable compute.googleapis.com storage.googleapis.com`
- 6 `gcloud compute regions list --filter="name~^asia-south"` — how many zones does `asia-south1` have?
- 7 Create a bucket: `gcloud storage buckets create gs://$PROJECT_ID-shipments --location=asia-south1 --uniform-bucket-level-access --public-access-prevention`
- 8 Console check: Navigation menu → Cloud Storage → Buckets → your bucket → Configuration tab — same location and public-access setting. Then TEARDOWN: `gcloud storage rm -r gs://$PROJECT_ID-shipments`

Console route for steps 5 and 7: APIs & Services → Enable APIs; Cloud Storage → Buckets → Create. Expected results on the next slide.

What you should have seen

DEBRIEF · LAB 1

```
$ gcloud services enable compute.googleapis.com storage.googleapis.com
Operation "operations/acf.p2-734912580216-..." finished successfully.
$ gcloud storage buckets create gs://$PROJECT_ID-shipments --
location=asia-south1 \
--uniform-bucket-level-access --public-access-prevention
Creating gs://acme-store-dev-shipments/...
$ gcloud storage buckets describe gs://$PROJECT_ID-shipments \
--format="value(location,iamConfiguration.publicAccessPrevention)"
ASIA-SOUTH1 enforced
```

Console check

Cloud Storage → Buckets → <project>-shipments → Configuration:

- Location: asia-south1 (Region)
- Access control: Uniform
- Public access: Not public (prevention enforced)

Same facts the CLI printed — one API, two doors.

Check yourself: asia-south1 has 3 zones · your projectNumber is a 12-digit number, not the ID · the bucket location prints in CAPITALS because the API stores it that way · the console shows the same values · teardown removes the bucket and its contents in one command.

THE RULE A working project = correct config, enabled APIs, one resource you can create, read and delete.

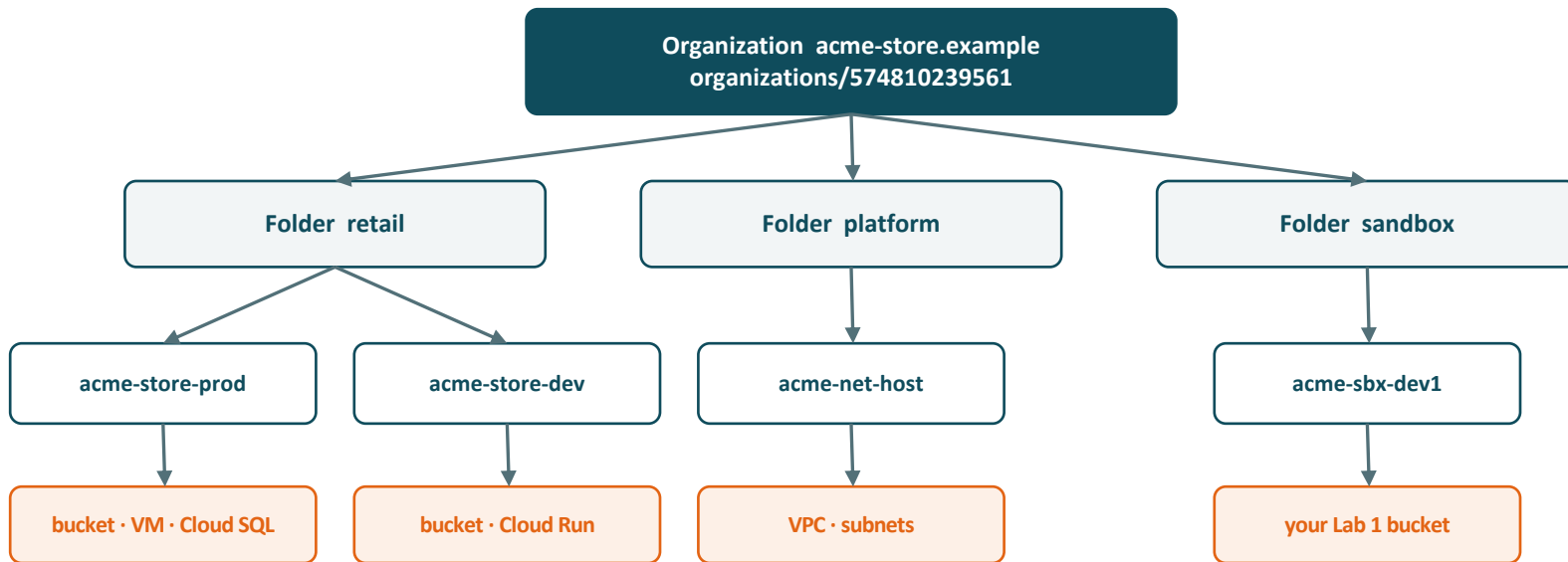
03

Where things live — the resource hierarchy

Organization, folders, projects and resources form a tree. Policy flows down it — and understanding that tree explains most 'why can't I...' questions.

- 01 What a cloud provider actually gives you
- 02 Getting to the keyboard
- 03 Where things live — the resource hierarchy**
- 04 Who can do what — identity and IAM
- 05 Paying for it — billing, cost and quotas
- 06 Who did what — audit logs

Organization → folders → projects → resources



Nodes: organization (1) · folders (optional, nest up to 10 deep) · projects (hold resources, billed) · resources (what you create)

THE RULE Only projects hold resources. Everything above a project exists to organise ownership and policy.

The unit of isolation, quota and billing — with three names

Identifier	Example	Set by	Changeable?	Where you see it
Project ID	acme-store-dev	you, at creation (globally unique)	never	gcloud, URLs, gs:// paths
Project number	734912580216	Google	never	SA emails, errors, quotas, billing export
Project name	Acme Store (dev)	you	any time	console header only

Isolation

Every resource belongs to exactly one project. Access across projects must be granted explicitly.

Quota

CPUs per region, API requests per minute, buckets — all counted per project. Noisy neighbours stay in their own project.

Billing

One project ↔ one billing account. Every line on the invoice carries a project ID. No billing account = no paid APIs.

THE RULE When in doubt, make a new project. Isolation, quota and cost boundaries all come for free.

One project for everything, or one per environment?

WEAK

Project: acme-store

- dev, staging and prod VMs side by side
- 40 people with roles/editor because 'they need to deploy'
- A load test in dev consumes the region's CPU quota; prod cannot scale
- Finance asks 'what does prod cost?' — nobody can answer
- Deleting a dev bucket deletes the wrong one

STRONG

Folder: retail

acme-store-dev · acme-store-stg · acme-store-prod

- Developers: Editor in dev, Viewer in prod
- Prod deploys via a service account only
- Quotas per project — dev load tests cannot touch prod
- Bill splits by project automatically; folder rolls it up
- Org policy on the folder applies to all three at once

THE RULE One project per environment (and often per team) is the default, not the exception.

POLICY INHERITANCE

IAM flows down the tree and only ever adds — it cannot subtract

Organization
roles/viewer → group:all-engineers@



Folder retail
roles/editor → group:retail-devs@



Project acme-store-dev
roles/storage.admin → sa:deployer@

EFFECTIVE on the project =
viewer (all-engineers) + editor (retail-devs) + storage.admin (deployer)

```
$ gcloud projects get-ancestors acme-store-dev
```

ID	TYPE
acme-store-dev	project
318842071119	folder
574810239561	organization

```
$ gcloud projects get-iam-policy acme-store-dev \
--format="table(bindings.role,bindings.members)"
```

ROLE	MEMBERS
roles/storage.admin	serviceAccount:deployer@acme-store-dev.iam.gserviceaccount.com
roles/owner	user:dev1@acme-store.example

inherited bindings are NOT shown here — check each ancestor

THE RULE Effective access = the union of every binding from the resource up to the organization.

Preventive guardrails: what may exist, regardless of who asks

Constraint (legacy name · managed name)	Type	What it prevents
constraints/iam.disableServiceAccountKeyCreation · iam.managed.disableServiceAccountKeyCreation	boolean	creating downloadable service-account keys
constraints/compute.vmExternalIpAccess · compute.managed.vmExternalIpAccess	list	VMs with public IPv4 addresses (allow-list)
constraints/gcp.resourceLocations	list	resources outside approved regions (residency)
constraints/iam.allowedPolicyMemberDomains	list	granting IAM to identities outside your domains
constraints/compute.requireOsLogin · compute.managed.requireOsLogin	boolean	SSH via metadata keys instead of IAM-backed OS Login
constraints/sql.restrictPublicIp · sql.managed.restrictPublicIp	boolean	Cloud SQL instances with a public IP
constraints/compute.skipDefaultNetworkCreation	boolean	the auto-created 'default' VPC in new projects

THE RULE IAM decides who; org policy decides what is possible at all. Guardrails beat guidelines.

Reading the effective policy, and what a guardrail looks like when it bites

```
$ gcloud resource-manager org-policies describe \
  constraints/iam.disableServiceAccountKeyCreation --project=acme-
store-dev --effective
booleanPolicy:
  enforced: true
constraint: constraints/iam.disableServiceAccountKeyCreation

$ gcloud resource-manager org-policies list --project=acme-store-dev --
show-unset | head -4
CONSTRAINT                                LIST_POLICY
BOOLEAN_POLICY  ETAG
constraints/iam.disableServiceAccountKeyCreation  -      SET
BwYt...
constraints/compute.vmExternalIpAccess           SET      -
BwYt...
```

DENIED

```
$ gcloud iam service-accounts keys create k.json --
iam-account=deployer@acme-store-
dev.iam.gserviceaccount.com
```

ERROR: (gcloud.iam.service-accounts.keys.create)
 FAILED_PRECONDITION: Key creation is not
 allowed on this service account.

FAILED_PRECONDITION = org policy.
 PERMISSION_DENIED = IAM.

Setting a policy (trainer demo, needs roles/orgpolicy.policyAdmin): `gcloud resource-manager org-policies enable-enforce constraints/iam.disableServiceAccountKeyCreation --folder=318842071119`

THE RULE FAILED_PRECONDITION means a guardrail, not a missing role. Read the effective policy before asking for access.

Think-pair-share

- 1 Arjun is roles/owner on folder retail and nothing on acme-store-prod. Can he delete a VM in acme-store-prod?
- 2 Arjun is Owner on acme-store-dev. The folder enforces constraints/compute.vmExternallpAccess with an empty allow-list. Can he give his dev VM a public IP?
- 3 Priya has roles/viewer at the organization. A colleague's sandbox project has no binding for her. Can she read its buckets?

Answers in the speaker notes — decide in pairs first.

Read your ancestry

HANDS-ON · LAB 2

Scenario: Northwind's platform team says your sandbox 'inherits the standard guardrails'. Verify that claim.

- 1 `gcloud projects describe $PROJECT_ID --format="table(projectId,projectNumber,name,parent.type,parent.id)"`
- 2 `gcloud projects get-ancestors $PROJECT_ID` — write down the folder and organization IDs.
- 3 `gcloud projects get-iam-policy $PROJECT_ID --format="table(bindings.role,bindings.members)"` — which roles do YOU hold directly here?
- 4 `gcloud resource-manager org-policies list --project=$PROJECT_ID` — policies set AT the project (often none).
- 5 `gcloud resource-manager org-policies describe constraints/iam.disableServiceAccountKeyCreation --project=$PROJECT_ID --effective`
- 6 Try to trip it: `gcloud iam service-accounts create lab2-probe && gcloud iam service-accounts keys create /tmp/k.json --iam-account=lab2-probe@$PROJECT_ID.iam.gserviceaccount.com`
- 7 Record the error type (FAILED_PRECONDITION vs success). TEARDOWN: `gcloud iam service-accounts delete lab2-probe@$PROJECT_ID.iam.gserviceaccount.com --quiet; rm -f /tmp/k.json`

What the tree told you

DEBRIEF · LAB 2

```
$ gcloud projects get-ancestors $PROJECT_ID
ID          TYPE
acme-sbx-dev1  project
902114377825  folder      <- 'sandbox' folder, shared by the whole cohort
574810239561  organization

$ gcloud resource-manager org-policies describe \
  constraints/iam.disableServiceAccountKeyCreation --project=$PROJECT_ID --effective
booleanPolicy:
  enforced: true

$ gcloud iam service-accounts keys create /tmp/k.json --iam-account=lab2-probe@$PROJECT_ID.iam.gserviceaccount.com
ERROR: (gcloud.iam.service-accounts.keys.create) FAILED_PRECONDITION: Key creation is not allowed on this service
account.
```

Check yourself: same folder ID as your neighbour · project-level org-policy list is empty but --effective shows enforced: true · the error is FAILED_PRECONDITION, not PERMISSION_DENIED · the probe service account is deleted.

04

Who can do what — identity and IAM

Who is asking, what are they allowed to do, and on which resource.
Identity types, roles, bindings — and how to read a policy cold.

- 01 What a cloud provider actually gives you
- 02 Getting to the keyboard
- 03 Where things live — the resource hierarchy
- 04 Who can do what — identity and IAM**
- 05 Paying for it — billing, cost and quotas
- 06 Who did what — audit logs

Four kinds of principal, and how each one proves who it is

Principal	Identifier format	Who/what it is	Typical use
User account	user:priya@acme-store.example	a person in Cloud Identity / Workspace	console, gcloud, break-glass
Google group	group:retail-devs@acme-store.example	a set of users, managed in Workspace/Cloud Identity	the thing you bind roles to
Service account	serviceAccount:deployer@acme-store-dev.iam.gserviceaccount.com	an identity for code; lives in a project	VMs, Cloud Run, CI pipelines
Federated workload	principal://iam.googleapis.com/projects/734912580216/locations/global/workloadIdentityPools/github/subject/repo:acme/store	an external identity trusted via a Workload Identity Pool	GitHub Actions, AWS, on-prem, GKE pods
Domain / all authenticated	domain:acme-store.example · allAuthenticatedUsers · allUsers	everyone in a domain / any Google account / anyone	public buckets (rarely), org-wide viewers

THE RULE Bind roles to groups for people and to service accounts for code. Never to individual users, never to keys.

Keys are the anti-pattern; attach, impersonate or federate instead

WEAK

```
$ gcloud iam service-accounts keys create key.json --iam-account=deployer@...
```

key.json committed to the repo 'for the CI job'. Valid until someone remembers to rotate it (never). Audit log shows 'deployer@' — not which human or which pipeline.

STRONG

Attach: `gcloud run deploy ... --service-account=deployer@...`

Impersonate: `gcloud storage ls --impersonate-service-account=deployer@...`

Federate: GitHub OIDC token → Workload Identity Pool → `deployer@...`

Every token lives ≤ 1 h; the audit log records the caller AND the SA.

```
$ gcloud projects add-iam-policy-binding acme-store-dev --member=user:priya@acme-store.example \
  --role=roles/iam.serviceAccountTokenCreator --condition=None
$ gcloud storage ls --impersonate-service-account=deployer@acme-store-dev.iam.gserviceaccount.com
WARNING: This command is using service account impersonation. All API calls will be executed as [deployer@acme-store-dev.iam.gserviceaccount.com].
gs://acme-store-dev-shipments/
```

THE RULE If you are about to download a service-account key, stop and pick attach, impersonate or federate.

A policy is a list of bindings: a role, its members, optionally a condition

```
$ gcloud projects get-iam-policy acme-store-dev --format=json
{
  "bindings": [
    { "role": "roles/viewer",
      "members": [ "group:retail-devs@acme-store.example" ] },
    { "role": "roles/storage.objectViewer",
      "members": [ "serviceAccount:reporter@acme-store-dev.iam.gserviceaccount.com" ],
      "condition": { "title": "invoices-only",
                    "expression": "resource.name.startsWith('projects/_/buckets/acme-store-invoices')" } },
    { "role": "roles/owner", "members": [ "user:dev1@acme-store.example" ] }
  ],
  "etag": "BwYtR2k9v1M=", "version": 3
}
```

Principal — the who: user:, group:, serviceAccount:, principal://

Role — a named set of permissions (roles/storage.objectViewer)

Permission — service.resource.verb, e.g. storage.objects.get

Binding — one role → many principals, plus an optional condition

Policy — all bindings on ONE resource; inherited bindings live on the ancestors

THE RULE Who (principal) can do what (role → permissions) on which resource (where the policy sits).

Basic, predefined and custom — and when each is appropriate

	Basic	Predefined	Custom
Examples	roles/owner, roles/editor, roles/viewer	roles/storage.objectViewer, roles/run.invoker, roles/logging.viewer	projects/acme-store-dev/roles/shipmentsReader
Scope of permissions	every service in the project	one service, one job	exactly the permissions you list
Maintained by	Google (never changes)	Google (grows with the service)	you (frozen until you edit it)
Appropriate when	personal sandbox; the project has one user	almost always — the default	no predefined role fits and the gap matters
Risk	far too broad; Editor can delete anything	low; still review what a role contains	drift: new API features silently excluded

```
$ gcloud iam roles describe roles/storage.objectViewer --format="value(includedPermissions)"
resourceManager.projects.get;resourceManager.projects.list;storage.managedFolders.get;storage.managedFolders.list;storage.objects.get;storage.objects.list
```

THE RULE Predefined by default. Basic only in a one-person sandbox. Custom only when you can name the gap.

Who can do what?

DRILL

```
# folder 318842071119 policy
roles/viewer    -> group:all-engineers@acme-store.example
roles/editor    -> group:retail-devs@acme-store.example

# project acme-store-prod policy
roles/owner      -> group:retail-leads@acme-store.example
roles/storage.objectViewer -> serviceAccount:reporter@acme-store-
prod.iam.gserviceaccount.com
  condition: resource.name.startsWith('projects/_/buckets/acme-store-
invoices')
roles/run.invoker      -> allUsers    (on service 'storefront' only)

# group membership: all-engineers = {priya, arjun, dev1}; retail-devs =
{arjun}; retail-leads = {dev1}
```

1

Can priya list buckets in acme-store-prod?

2

Can arjun delete a VM in acme-store-prod?

3

Can reporter@ read gs://acme-store-static/logo.png?

4

Can arjun grant himself Owner on prod?

5

Can an anonymous user call the storefront service?

6

Who can add a binding on the folder?

Answers on the next slide.

Who can do what — answers

REVEAL

#	Question	Answer	Why
1	priya lists buckets in prod	Yes	roles/viewer on the folder is inherited; viewer includes storage.buckets.list
2	arjun deletes a VM in prod	Yes	roles/editor from the folder (retail-devs) is inherited; editor includes compute.instances.delete
3	reporter@ reads acme-store-static/logo.png	No	the condition limits objectViewer to the acme-store-invoices bucket
4	arjun grants himself Owner	No	editor lacks resourceManager.projects.setIamPolicy; only Owner/projectIamAdmin can change IAM
5	anonymous calls storefront	Yes	allUsers with run.invoker on that service — public by design (and worth an org policy)
6	who can add a folder binding	Unknown	needs the organization policy (folder IAM admin is usually granted there) — not shown

THE RULE To answer 'can X do Y', read the policy on the resource AND every ancestor, then apply conditions.

Build the smallest role that does the job

```
$ gcloud iam roles create shipmentsReader --project=acme-store-dev \
  --title="Shipments Reader" --description="List and read objects, nothing
else" \
  --permissions=storage.objects.get,storage.objects.list --stage=GA
Created role [shipmentsReader].
etag: BwYtS7pQ2hE=
includedPermissions:
- storage.objects.get
- storage.objects.list
name: projects/acme-store-dev/roles/shipmentsReader
stage: GA
title: Shipments Reader

$ gcloud storage buckets add-iam-policy-binding gs://acme-store-dev-
shipments \
  --member=serviceAccount:reporter@acme-store-dev.iam.gserviceaccount.com
\
  --role=projects/acme-store-dev/roles/shipmentsReader
```

- Start from the predefined role's permission list
- Remove, don't add: fewer permissions than the predefined role
- Stage: ALPHA/BETA while testing, GA when in use
- Custom role names: projects/.../roles/ID or organizations/.../roles/ID
- Re-review custom roles every time the service ships a new feature

THE RULE A custom role is a maintenance commitment. Make it only when a predefined role is provably too wide.

Least privilege in practice

HANDS-ON · LAB 3

Scenario: Northwind's reporting job must READ shipment files and nothing else. Prove it can, and prove it cannot write.

- 1 Recreate the bucket and put a file in it: `gcloud storage buckets create gs://$PROJECT_ID-shipments --location=asia-south1 -b --pap && echo hello | gcloud storage cp - gs://$PROJECT_ID-shipments/s1.txt`
- 2 Create the identity: `gcloud iam service-accounts create reporter --display-name="Shipments reporter"; export SA=reporter@$PROJECT_ID.iam.gserviceaccount.com`
- 3 Let yourself impersonate it: `gcloud iam service-accounts add-iam-policy-binding $SA --member=user:$(gcloud config get-value account) --role=roles/iam.serviceAccountTokenCreator`
- 4 Grant read on the bucket only: `gcloud storage buckets add-iam-policy-binding gs://$PROJECT_ID-shipments --member=serviceAccount:$SA --role=roles/storage.objectViewer`
- 5 Prove read works, write fails (wait ~60 s first): `gcloud storage cat gs://$PROJECT_ID-shipments/s1.txt --impersonate-service-account=$SA ; echo x | gcloud storage cp - gs://$PROJECT_ID-shipments/s2.txt --impersonate-service-account=$SA`
- 6 Dump the project policy: `gcloud projects get-iam-policy $PROJECT_ID --format=json` — is reporter listed? Why not?
- 7 Dump the bucket policy: `gcloud storage buckets get-iam-policy gs://$PROJECT_ID-shipments --format=json`
- 8 Custom role: `gcloud iam roles create shipmentsReader --project=$PROJECT_ID --permissions=storage.objects.get,storage.objects.list --stage=GA`, swap the binding, re-test step 5. **TEARDOWN** on the next slide.

What least privilege looks like in the output

DEBRIEF · LAB 3

```
$ gcloud storage cat gs://$PROJECT_ID-shipments/s1.txt --impersonate-service-account=$SA
WARNING: This command is using service account impersonation. All API calls will be executed as [reporter@...].
hello
$ echo x | gcloud storage cp - gs://$PROJECT_ID-shipments/s2.txt --impersonate-service-account=$SA
ERROR: (gcloud.storage.cp) User [reporter@acme-sbx-dev1.iam.gserviceaccount.com] does not have permission to
access b instance [acme-sbx-dev1-shipments] (or it may not exist): reporter@... does not have storage.objects.create access
to the Google Cloud Storage object. Permission 'storage.objects.create' denied on resource (or it may not exist).
$ gcloud storage buckets get-iam-policy gs://$PROJECT_ID-shipments --format="table(bindings.role,bindings.members)"
ROLE                                MEMBERS
roles/storage.legacyBucketOwner    projectEditor:acme-sbx-dev1;projectOwner:acme-sbx-dev1
roles/storage.legacyBucketReader    projectViewer:acme-sbx-dev1
roles/storage.objectViewer          serviceAccount:reporter@acme-sbx-dev1.iam.gserviceaccount.com
```

```
TEARDOWN gcloud storage rm -r gs://$PROJECT_ID-shipments && gcloud iam service-accounts delete $SA --quiet \
&& gcloud iam roles delete shipmentsReader --project=$PROJECT_ID
```

THE RULE Bind the narrowest role on the narrowest resource. The project policy stays small; the audit trail stays readable.

05

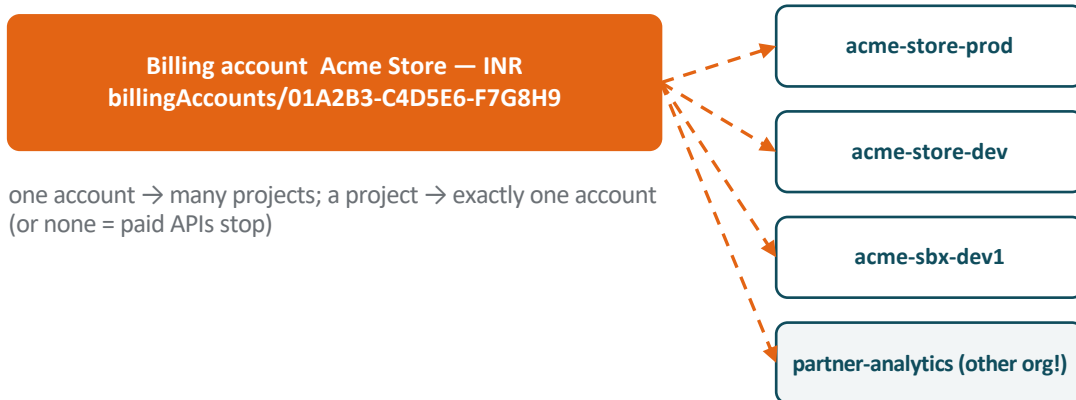
Paying for it — billing, cost and quotas

How money flows from resources to invoices, how to see it coming, how to attribute it, and how to pay less for what you already run.

- 01 What a cloud provider actually gives you
- 02 Getting to the keyboard
- 03 Where things live — the resource hierarchy
- 04 Who can do what — identity and IAM
- 05 Paying for it — billing, cost and quotas**
- 06 Who did what — audit logs

BILLING ACCOUNTS

A billing account pays for projects; it is not part of the resource tree



one account → many projects; a project → exactly one account
(or none = paid APIs stop)

Role	Can
Billing Account Administrator	payment method, budgets, link any project, export
Billing Account User	link projects to the account
Billing Account Viewer	see cost and usage reports
Project Billing Manager	attach / detach the project (project side)

```
$ gcloud billing projects describe acme-store-dev
billingAccountName: billingAccounts/01A2B3-C4D5E6-F7G8H9
billingEnabled: true
name: projects/acme-store-dev/billingInfo
projectId: acme-store-dev
```

THE RULE Billing is a separate tree. Who can see the bill is a billing-account role, not a project role.

A budget does not stop spending — it tells you, early

```
$ gcloud billing budgets create --billing-account=01A2B3-C4D5E6-F7G8H9 \
  --display-name="acme-store-dev monthly" --budget-amount=20000INR \
  --filter-projects=projects/734912580216 \
  --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 \
  --threshold-rule=percent=1.0,basis=forecasted-spend
Created budget [billingAccounts/01A2B3-C4D5E6-F7G8H9/budgets/9c1f0d7e-...].

$ gcloud billing budgets list --billing-account=01A2B3-C4D5E6-F7G8H9 \
  --
format="table(displayName,amount.specifiedAmount.units,thresholdRules.len())"

```

DISPLAY_NAME	UNITS	THRESHOLD_RULES
acme-store-dev monthly	20000	3

- Amount: fixed, or last month's spend
- Thresholds: % of actual OR forecasted spend
- Filters: projects, services, labels, folders
- Delivery: email to billing admins, Monitoring channels, Pub/Sub
- A budget never stops anything by itself

STRONG

Three rules per budget: 50% actual (heads-up) · 90% actual (act now) · 100% forecasted (this month will overrun — fires early).

THE RULE Every project gets a budget with a forecasted-spend rule on day one. Alerts you never set never fire.

Labels turn one invoice into a per-team, per-app, per-environment report

Key	Values	Answers the question
team	retail, platform, data	who owns it (showback)
app	storefront, shipments, warehouse	what it is for
env	dev, stg, prod	may I switch it off at night?
cost-center	cc-4410, cc-4420	which budget line (chargeback)

```
$ gcloud storage buckets update gs://acme-store-dev-shipments \
  --update-labels=team=retail,app=shipments,env=dev
Updating gs://acme-store-dev-shipments/...
$ gcloud storage buckets describe gs://acme-store-dev-shipments \
  --format="yaml (labels)"
labels:
  app: shipments
  env: dev
  team: retail
```

```
-- billing export in BigQuery: cost per team, this month
SELECT (SELECT value FROM UNNEST(labels) WHERE key='team') AS team, ROUND(SUM(cost),2) AS cost
FROM `acme-billing.billing.gcp_billing_export_v1_01A2B3_C4D5E6_F7G8H9`
WHERE invoice.month = '202609' GROUP BY team ORDER BY cost DESC
team      cost
retail    184230.55
platform  61920.10
```

THE RULE Four label keys, lowercase, applied by automation. The 'NULL' row in the cost report is your labelling debt.

COMMITTED USE AND SUSTAINED USE DISCOUNTS

Commit for what you know you'll run; don't plan around sustained-use

	Sustained use (SUD)	Resource-based CUD	Compute Flexible CUD
What you commit	nothing — automatic	vCPU + RAM of one family, in one region, 1 or 3 yrs	an hourly \$ spend, 1 or 3 yrs
Applies to	N1, N2, N2D, C2, M1, M2 only (legacy)	the committed family/region (Compute Engine, GKE nodes)	Compute Engine, GKE, Cloud Run — any region/family
Flexibility	n/a	low — locked to family and region	high — moves with your workload
Use when	you happen to run legacy families 24x7	steady baseline you can name exactly	steady spend you can name, shape may change
Does not cover	E2, C3, C4, N4, Tau and newer	other regions, other families	storage, network egress, other services

Decision: measure 3 months of on-demand spend → commit to ~70–80% of the steady floor with a Flexible CUD → add resource-based CUDs only for a family you will not change. Never commit to the peak.

THE RULE Commit to the floor you can prove, flexibly first. Treat sustained-use discounts as a bonus on legacy VMs, not a plan.

Quotas protect you and Google; you can see them, and most can be raised

```
$ gcloud quotas info list --service=compute.googleapis.com --project=acme-store-dev \
  --filter="quotaId:CPUS-per-project-region" --
format="table(quotaId,dimensions,quotaValue)"
QUOTA_ID                DIMENSIONS                QUOTA_VALUE
CPUS-per-project-region  region=asia-south1        24
CPUS-per-project-region  region=asia-south2        24

$ gcloud quotas preferences create --service=compute.googleapis.com --project=acme-
store-dev \
  --quota-id=CPUS-per-project-region --dimensions=region=asia-south1 --preferred-
value=96 \
  --preference-id=cpus-asia-south1-96 --justification="Load test for shipments
launch, Oct 2026"
Created preference [projects/acme-store-dev/locations/global/quotaPreferences/cpus-
asia-south1-96].
```

- Rate quota: per-minute, self-resets (API calls)
- Allocation quota: a count (CPUs, IPs, buckets)
- Limit: fixed, cannot be raised
- Error: 429 RESOURCE_EXHAUSTED or QUOTA_EXCEEDED
- Ask early — increases are reviewed, not instant

Console: IAM & Admin → Quotas & System Limits → filter by service → Edit quota. Same request, same review.

THE RULE Check the quota for every region you'll deploy to before the launch week, and file increases with a justification.

WHICH CONTROL?

Match the problem to the mechanism

- 1 "We found out about the ₹9 lakh overrun when the invoice arrived."
- 2 "Finance wants to know what each team spends, but everything is in one project."
- 3 "We run ~200 vCPUs 24×7 but we're migrating from N2 to C3 next quarter."
- 4 "The load test keeps dying with 429 RESOURCE_EXHAUSTED at 24 vCPUs."
- 5 "Someone created a bucket in us-east1; our data must stay in India."

Options: budget · labels + export · Flexible CUD · quota increase · org policy — answers in the speaker notes.

Cost controls on your project

HANDS-ON · LAB 4

Scenario: Northwind finance wants shipments costs visible per team, and a warning before the sandbox overruns ₹20,000/month.

- 1 Find your billing link: `gcloud billing projects describe $PROJECT_ID` — record the billing account ID and `billingEnabled`.
- 2 Label for showback: recreate the bucket (`gcloud storage buckets create gs://$PROJECT_ID-shipments --location=asia-south1 -b --pap`), then `gcloud storage buckets update gs://$PROJECT_ID-shipments --update-labels=team=retail,app=shipments,env=sandbox` and read the labels back.
- 3 Quota check: `gcloud quotas info list --service=compute.googleapis.com --project=$PROJECT_ID --filter="quotaid:CPUS-per-project-region" --format="table(quotaid,dimensions,quotaValue)"` — what is your asia-south1 CPU quota? Write (do NOT run) the preferences create command for 48.
- 4 Budget: try `gcloud billing budgets create --billing-account=<ID> --display-name="$PROJECT_ID monthly" --budget-amount=20000INR --filter-projects=projects/$PROJECT_NUMBER --threshold-rule=percent=0.5 --threshold-rule=percent=1.0,basis=forecasted-spend` — note the result (success or `PERMISSION_DENIED` and which role is missing).
- 5 In the console: Billing → Cost breakdown — filter by label `team=retail`. TEARDOWN: `gcloud storage rm -r gs://$PROJECT_ID-shipments` (and delete the budget if you created one: `gcloud billing budgets delete <name>`).

What you could see, and what needed a billing role

DEBRIEF · LAB 4

```
$ gcloud billing projects describe $PROJECT_ID
billingAccountName: billingAccounts/01A2B3-C4D5E6-F7G8H9
billingEnabled: true
$ gcloud storage buckets describe gs://$PROJECT_ID-shipments --format="yaml(labels)"
labels: {app: shipments, env: sandbox, team: retail}
$ gcloud quotas info list --service=compute.googleapis.com --project=$PROJECT_ID --filter="quotaId:CPUS-per-project-region" \
  --format="table(quotaId,dimensions,quotaValue)" | head -3
QUOTA_ID          DIMENSIONS          QUOTA_VALUE
CPUS-per-project-region  region=asia-south1  24
$ gcloud billing budgets create --billing-account=01A2B3-C4D5E6-F7G8H9 --display-name="$PROJECT_ID monthly" ...
ERROR: (gcloud.billing.budgets.create) PERMISSION_DENIED: The caller does not have permission
[billing.budgets.create] on billingAccounts/01A2B3-C4D5E6-F7G8H9
```

Check yourself: labels read back in alphabetical key order · CPU quota is per region · the budget permission is on the billing account, not the project.

THE RULE Labels and quotas are project-side and yours; budgets and discounts are billing-account-side and someone else's — until you ask.

06

Who did what — audit logs

Every change you made today was recorded, with your identity attached. Four audit log types, where they live, how long they stay, and how to ask 'who did that?'.

- 01 What a cloud provider actually gives you
- 02 Getting to the keyboard
- 03 Where things live — the resource hierarchy
- 04 Who can do what — identity and IAM
- 05 Paying for it — billing, cost and quotas
- 06 Who did what — audit logs**

Four log types — three always on, one you must switch on

Log type	Records	On by default?	Bucket · retention	Role to read	Log name
Admin Activity	API calls that change configuration or metadata	always (cannot disable)	_Required · 400 days, fixed	roles/logging.viewer	cloudaudit.googleapis.com/activity
Data Access	reads of config; reads & writes of user data	no (except BigQuery)	_Default · 30 days (1–3650 configurable)	roles/logging.privateLog Viewer	cloudaudit.googleapis.com/data_access
System Event	Google-initiated changes (live migration, etc.)	always (cannot disable)	_Required · 400 days, fixed	roles/logging.viewer	cloudaudit.googleapis.com/system_event
Policy Denied	requests denied by a security policy	always (excludable)	_Default · 30 days	roles/logging.viewer	cloudaudit.googleapis.com/policy

Data Access logs cost ingestion; Admin Activity logs are free and cannot be turned off — which is why 'who changed IAM?' is always answerable.

THE RULE Configuration changes are always logged. Data reads are logged only where you switched them on.

Five fields answer 'who did what, to which resource, from where, when'

```
{
  "logName": "projects/acme-store-dev/logs/cloudaudit.googleapis.com%2Factivity",
  "timestamp": "2026-09-06T05:41:17.902Z",
  "protoPayload": {
    "authenticationInfo": { "principalEmail": "dev1@acme-store.example" },
    "methodName": "storage.buckets.create",
    "resourceName": "projects/_/buckets/acme-store-dev-shipments",
    "serviceName": "storage.googleapis.com",
    "requestMetadata": { "callerIp": "35.201.14.77",
      "callerSuppliedUserAgent": "google-cloud-sdk gcloud/583.0.0
command/gcloud.storage.buckets.create ..." },
    "authorizationInfo": [ { "permission": "storage.buckets.create", "granted": true } ]
  },
  "resource": { "type": "gcs_bucket", "labels": { "bucket_name": "acme-store-dev-
shipments", "project_id": "acme-store-dev" } },
  "severity": "NOTICE"
}
```

WHO principalEmail

WHAT methodName

ON WHAT resourceName

FROM WHERE callerIp, user agent

WHEN timestamp

ALLOWED? authorizationInfo.granted;
status.code 7 = denied

THE RULE principalEmail + methodName + resourceName is the sentence: who did what to which thing.

The same filter works in Logs Explorer and in gcloud logging read

```
$ gcloud logging read \  
  'logName="projects/acme-store-dev/logs/cloudaudit.googleapis.com%2Factivity"  
  AND protoPayload.serviceName="storage.googleapis.com"' \  
  --freshness=6h --limit=5 \  
  --format="table(timestamp,protoPayload.authenticationInfo.principalEmail,protoPayload.methodName,protoPayload.resourceName)"  
TIMESTAMP          PRINCIPAL_EMAIL          METHOD_NAME          RESOURCE_NAME  
2026-09-06T06:12:40.118Z  dev1@acme-store.example  storage.setIamPermissions  projects/_/buckets/acme-store-dev-shipments  
2026-09-06T05:41:17.902Z  dev1@acme-store.example  storage.buckets.create     projects/_/buckets/acme-store-dev-shipments  
  
# who touched IAM on the project today?  
$ gcloud logging read 'protoPayload.methodName="SetIamPolicy" AND resource.type="project"' --freshness=1d \  
  --format="value(timestamp,protoPayload.authenticationInfo.principalEmail)"  
2026-09-06T06:30:02.441Z  dev1@acme-store.example
```

Console: Logging → Logs Explorer → paste the same filter → Run query. Tip: click any field value in a result to add it to the query.

THE RULE Filter on logName + serviceName first, then narrow by principalEmail or methodName. Widen --freshness before you conclude 'nothing happened'.

Switched on per service — in the console, stored in the IAM policy

Console (the safe route)

- 1 IAM & Admin → Audit Logs → filter the table for Google Cloud Storage
- 2 Tick Data Read and Data Write (Admin Read is usually noise) → Save
- 3 The change is itself an Admin Activity entry (SetIamPolicy) — you are auditable too

logType	Records
ADMIN_READ	reading configuration/metadata (get bucket, get IAM)
DATA_READ	reading user data (download an object)
DATA_WRITE	writing user data (upload, delete an object)

```
$ gcloud projects get-iam-policy acme-store-dev --format="yaml(auditConfigs)"
auditConfigs:
- auditLogConfigs:
  - logType: DATA_READ
  - logType: DATA_WRITE
  service: storage.googleapis.com
# stored inside the project's IAM policy — that is why the console page is under
IAM & Admin
```

DANGER

`gcloud projects set-iam-policy` REPLACES the whole policy. A hand-edited file without bindings strips every role from the project. Use the console page for this one.

THE RULE Switch Data Access logs on per service in IAM & Admin → Audit Logs; verify with `gcloud`; never hand-edit the policy file.

Find yourself in the logs

HANDS-ON · LAB 5

Scenario: Northwind security asks 'who created that shipments bucket, and has anyone read the files?' Answer both from the logs.

1

Who created buckets today? `gcloud logging read 'logName="projects/$PROJECT_ID/logs/cloudaudit.googleapis.com%2Factivity" AND protoPayload.methodName="storage.buckets.create"' --freshness=1d --format="table(timestamp,protoPayload.authenticationInfo.principalEmail,protoPayload.resourceName)"`

2

Who touched IAM? Same command with `protoPayload.methodName="SetIamPolicy"` — find your Lab 3 binding change.

3

Console: IAM & Admin → Audit Logs → Google Cloud Storage → tick Data Read + Data Write → Save. Verify: `gcloud projects get-iam-policy $PROJECT_ID --format="yaml(auditConfigs)"`

4

Create data and read it: `gcloud storage buckets create gs://$PROJECT_ID-shipments --location=asia-south1 -b --pap; echo hello | gcloud storage cp - gs://$PROJECT_ID-shipments/s1.txt; gcloud storage cat gs://$PROJECT_ID-shipments/s1.txt`

5

Wait ~90 s, then find the read: `gcloud logging read 'logName="projects/$PROJECT_ID/logs/cloudaudit.googleapis.com%2Fdata_access" AND protoPayload.methodName="storage.objects.get"' --freshness=10m --format="table(timestamp,protoPayload.authenticationInfo.principalEmail,protoPayload.resourceName)"`

6

Open Logs Explorer in the console, paste the step-5 filter, look at the histogram. TEARDOWN: untick Data Read/Data Write in Audit Logs → Save, then `gcloud storage rm -r gs://$PROJECT_ID-shipments`

Two questions, two log types

DEBRIEF · LAB 5

```
$ gcloud logging read '... methodName="storage.buckets.create"' --freshness=1d --
format="table(timestamp,...principalEmail,...resourceName)"
TIMESTAMP                PRINCIPAL_EMAIL                RESOURCE_NAME
2026-09-06T08:02:11.530Z   dev1@acme-store.example        projects/_/buckets/acme-sbx-dev1-shipments
2026-09-06T06:55:48.007Z   dev1@acme-store.example        projects/_/buckets/acme-sbx-dev1-shipments
2026-09-06T05:41:17.902Z   dev1@acme-store.example        projects/_/buckets/acme-sbx-dev1-shipments

$ gcloud projects get-iam-policy $PROJECT_ID --format="yaml(auditConfigs)"
auditConfigs:
- auditLogConfigs: [{logType: DATA_READ}, {logType: DATA_WRITE}]
  service: storage.googleapis.com

$ gcloud logging read '... data_access ... methodName="storage.objects.get"' --freshness=10m ...
TIMESTAMP                PRINCIPAL_EMAIL                RESOURCE_NAME
2026-09-06T08:04:39.216Z   dev1@acme-store.example        projects/_/buckets/acme-sbx-dev1-shipments/objects/s1.txt
```

Check yourself: three bucket creates today (Labs 1, 3/4, 5) · the object read appears ONLY after Data Access logging was switched on · after teardown `yaml(auditConfigs)` prints nothing.

THE RULE Admin Activity answers 'who changed it'. Data Access answers 'who read it' — only if you switched it on first.

RECAP

The four questions you can now answer about any Google Cloud resource

Where does it live?

org → folder → project → resource
gcloud projects get-ancestors

What is allowed here?

org policy, inherited
org-policies describe --effective

Who can do what?

principals → roles → bindings
get-iam-policy · impersonate

Who pays, and how much?

billing account · labels · budgets · quotas
billing projects describe

Who did that?

Admin Activity always · Data Access opt-in
gcloud logging read

How do I reach it?

console · gcloud · client library
Cloud Shell, APIs enabled

THE RULE Hierarchy, guardrails, identity, money, evidence — check all five before you ship anything to a project.

The commands from today, in one place

Area	Commands
Setup	gcloud config list · gcloud config set project ID · gcloud auth list · gcloud services enable X.googleapis.com · gcloud services list --enabled
Hierarchy	gcloud projects describe ID · gcloud projects get-ancestors ID · gcloud resource-manager org-policies list --project=ID · ... describe CONSTRAINT --project=ID --effective
IAM	gcloud projects get-iam-policy ID --format=json · gcloud projects add-iam-policy-binding ID --member=group:G --role=roles/X · gcloud iam roles describe roles/X · gcloud iam roles create ID --project=P --permissions=... · gcloud iam service-accounts create NAME · --impersonate-service-account=SA
Storage	gcloud storage buckets create gs://B --location=L -b --pap · gcloud storage buckets update gs://B --update-labels=k=v · gcloud storage buckets get-iam-policy gs://B · gcloud storage rm -r gs://B
Billing & quotas	gcloud billing projects describe ID · gcloud billing budgets create --billing-account=A --budget-amount=N --threshold-rule=percent=0.5 · gcloud quotas info list --service=S --project=ID · gcloud quotas preferences create --service=S --quota-id=Q --preferred-value=N
Audit logs	gcloud logging read 'logName="projects/ID/logs/cloudaudit.googleapis.com%2Factivity"' --freshness=1d --limit=20 · Data Access on/off: console IAM & Admin → Audit Logs → service → Data Read / Data Write · verify: gcloud projects get-iam-policy ID --format="yaml(auditConfigs)"

Where this course leads

Build on it

- Compute Engine & networking (VPC, firewalls, load balancing)
- Cloud Run & GKE — deploying with an attached service account
- Terraform for the hierarchy, IAM and org policies you saw today

Do on Monday

- `gcloud config list` before every session
- Find one basic-role binding and replace it with a predefined role
- Put a forecasted-spend budget on your project
- Label every resource you own with `team/app/env`

Read

- docs.cloud.google.com/iam/docs/roles-overview
- docs.cloud.google.com/resource-manager/docs/organization-policy
- docs.cloud.google.com/logging/docs/audit
- docs.cloud.google.com/sdk/gcloud (reference for every command)

THE RULE Teardown before you leave: buckets, service accounts, the custom role, auditConfigs, any budget you created.